Hari Patel Project 2

Problem 1

```
# Project 2 – Problem 1
# Data: U.S. debt per capita
years := [1900, 1910, 1920, 1930, 1940, 1950, 1960, 1970, 1980, 1990, 2000, 2010, 2020, 2024];
debt := [16, 12, 228, 131, 325, 1688, 1572, 1814, 3985, 13000, 20163, 43737, 80885, 104599];
# Lagrange interpolation procedure
Lagrange := proc(X, Y, xval)
  local i, j, n, L, P;
  n := nops(X); P := 0;
  for i from 1 to n do
    L := 1;
    for j from 1 to n do
      if i ≠ j then
        L := L * (xval − X[j]) / (X[i] − X[j]);
      end if;
    end do;
    P := P + Y[i]*L;
  end do;
  P;
end proc;
# 1st-degree polynomial (2000, 2010)
X1 := [2000, 2010];
Y1 := [20163, 43737];
P1 := x → Lagrange(X1, Y1, x);
printf("1st-degree polynomial:\n");
expand(Lagrange(X1, Y1, x));      # show polynomial
printf("1st degree approx (2005): %.2f\n\n", evalf(P1(2005)));
# 3rd-degree polynomial (1990, 2000, 2010, 2020)
X3 := [1990, 2000, 2010, 2020];
Y3 := [13000, 20163, 43737, 80885];
P3 := x → Lagrange(X3, Y3, x);
printf("3rd-degree polynomial:\n");
expand(Lagrange(X3, Y3, x));      # show polynomial
printf("3rd degree approx (2005): %.2f\n\n", evalf(P3(2005)));
# 5th-degree polynomial (1980, 1990, 2000, 2010, 2020, 2024)
X5 := [1980, 1990, 2000, 2010, 2020, 2024];
Y5 := [3985, 13000, 20163, 43737, 80885, 104599];
P5 := x → Lagrange(X5, Y5, x);
printf("5th-degree polynomial:\n");
expand(Lagrange(X5, Y5, x));      # show polynomial
printf("5th degree approx (2005): %.2f\n\n", evalf(P5(2005)));
# Plots
with(plots) :
plot([seq([years[i], debt[i]], i = 1 ..nops(years))], style = point, symbol = solidcircle);
%;
plot([P1(x), P3(x), P5(x)], x = 1990 ..2020);
```

$$years := [1900, 1910, 1920, 1930, 1940, 1950, 1960, 1970, 1980, 1990, 2000, 2010, 2020, 2024]$$

$$debt := [16, 12, 228, 131, 325, 1688, 1572, 1814, 3985, 13000, 20163, 43737, 80885, 104599]$$

$$Lagrange := \mathbf{proc}(X, Y, xval)\ \mathbf{local}\ i, j, n, L, P;\ n := nops(X);\ P := 0;\ \mathbf{for}\ i\ \mathbf{to}\ n\ \mathbf{do}\ L := 1;\ \mathbf{for}\ j\ \mathbf{to}\ n\ \mathbf{do}\ \mathbf{if}\ i <> j\ \mathbf{then}\ L := L * (xval - X[j]) / (X[i] - X[j])\ \mathbf{end\ if}\ \mathbf{end\ do};\ P := P + Y[i] * L\ \mathbf{end\ do};\ P\ \mathbf{end\ proc}$$

$$X1 := [2000, 2010]$$

$$Y1 := [20163, 43737]$$

$$P1 := x \mapsto Lagrange(X1, Y1, x)$$

1st-degree polynomial:

$$\frac{11787\,x}{5} - 4694637$$

1st degree approx (2005): 31950.00

$$X3 := [1990, 2000, 2010, 2020]$$

$$Y3 := [13000, 20163, 43737, 80885]$$

$$P3 := x \mapsto Lagrange(X3, Y3, x)$$

3rd-degree polynomial:

$$-\frac{2837}{6000}x^3 + \frac{583811}{200}x^2 - \frac{90009538}{15}x + 4107738563$$

3rd degree approx (2005): 30075.94

$$X5 := [1980, 1990, 2000, 2010, 2020, 2024]$$

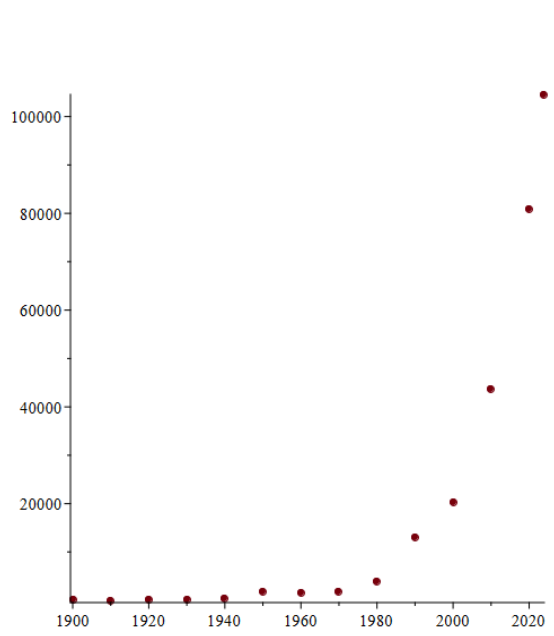
$$Y5 := [3985, 13000, 20163, 43737, 80885, 104599]$$

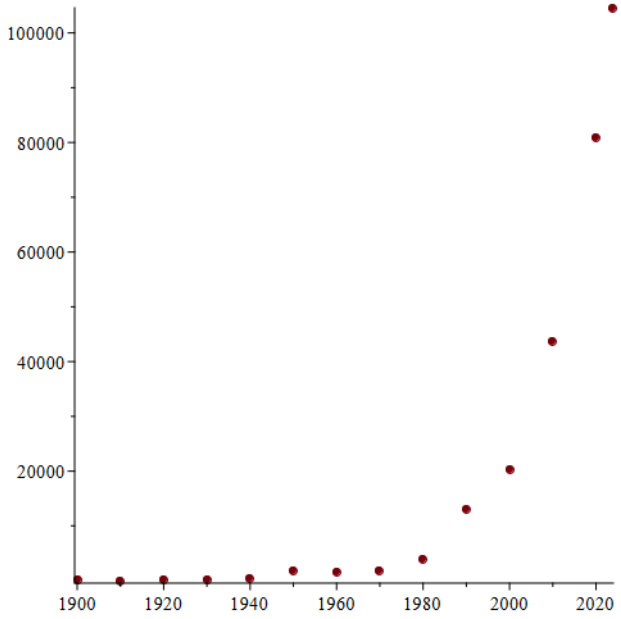
$$P5 := x \mapsto Lagrange(X5, Y5, x)$$

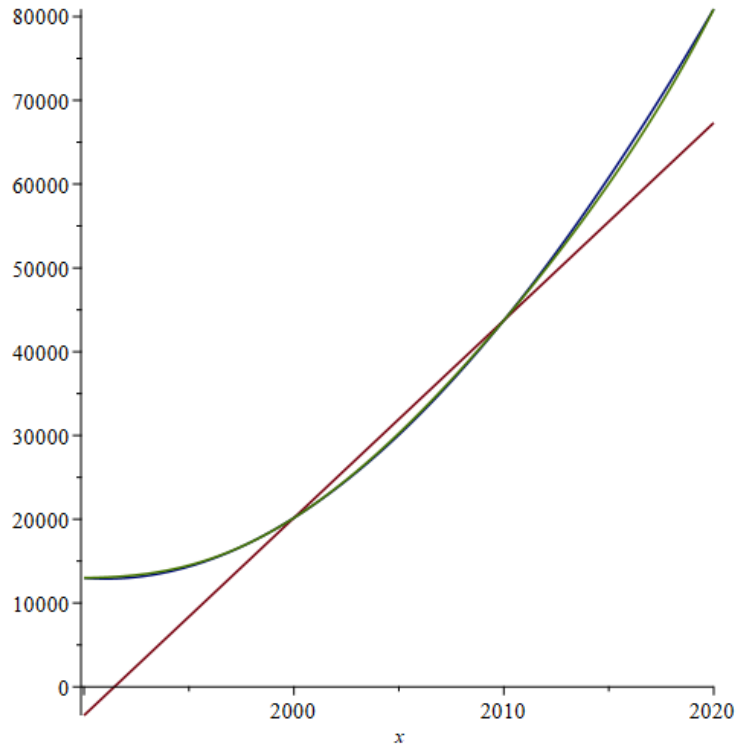
5th-degree polynomial:

$$\frac{15277133631713141}{39270}x - \frac{76248951259807}{196350}x^2 - \frac{18550761052936003}{119} - \frac{151948199}{3141600}x^4 + \frac{18959}{3927000}x^5 + \frac{1522235117317}{7854000}x^3$$

5th degree approx (2005): 30260.32







The best was the 3rd degree polynomial. The graph of the third was not as simplified as the first degree polynomial graph. It was better than the 5th degree polynomial graph due to the 5th degree trying too hard to fit to the points resulting in a less smooth graph.

Problem 2 The best possible error Bound was 1.934382

```
f := x -> x*exp(-x^2);
X := [-1, -1/2, 1, 9/4, 3];

 #Computing the 5 th derivative
f5 := simplify(diff(f(x), x$5)) :
printf("f^(5)(x) computed.\n");
# Finding the]max|f^(5)(x)| on [-1, 3]
maxval := 0 :
for i from -100 to 300 do
  xi := i/100;
  val := abs(evalf(subs(x = xi, f5)));
  if evalf(val) > evalf(maxval) then
    maxval := evalf(val);
  end if;
end do:
printf("Maximum |f^(5)(x)| on [-1,3] ≈ %.4f\n", maxval);
# Compute omega(x)
omega := (x - X[1])*(x - X[2])*(x - X[3])*(x - X[4])*(x - X[5]) :

# Find max |omega(x)| on [-1,3]
maxomega := 0 :
for i from -100 to 300 do
  xi := i/100;
  val := abs(evalf(subs(x = xi, omega)));
  if evalf(val) > evalf(maxomega) then
    maxomega := evalf(val);
  end if;
end do:
printf("Maximum |omega(x)| on [-1,3] ≈ %.4f\n", maxomega);

# Figuring outError bound
n := 4;
ErrorBound := evalf( (maxval/factorial(n + 1))*maxomega );
printf("Best possible error bound ≈ %.6f\n", ErrorBound);


plot(f(x), x = -1..3);
```

$$f := x \mapsto x \cdot e^{-x^2}$$

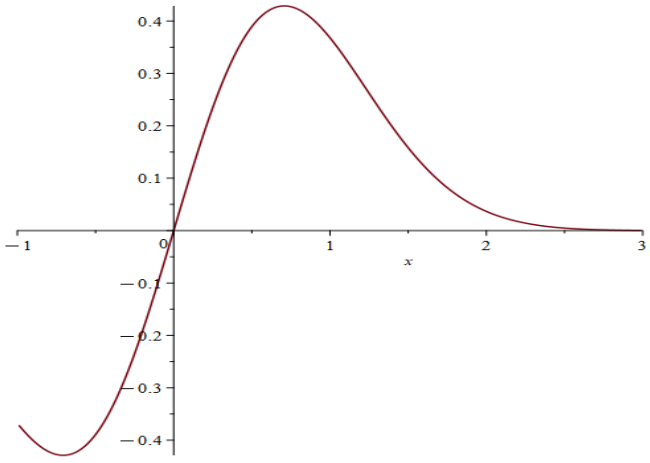
$$X := \left[-1, -\frac{1}{2}, 1, \frac{9}{4}, 3\right]$$

```
f^(5)(x) computed.
Maximum |f^(5)(x)| on [-1,3] &approx; 60.0000
Maximum |omega(x)| on [-1,3] &approx; 3.8688
```

$$n := 4$$

$$ErrorBound := 1.934381549$$

```
Best possible error bound &approx; 1.934382
```

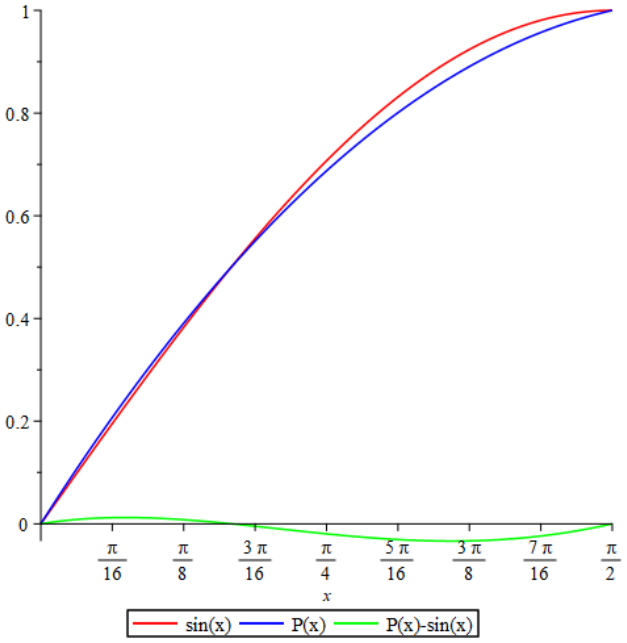


Problem 3

```
f := x → sin(x) :
Xvals := [0, Pi/6, Pi/2] :
Yvals := [evalf(f(Xvals[1])), evalf(f(Xvals[2])), evalf(f(Xvals[3]))] :
# Lagrange interpolation procedure
Lagrange := proc(X, Y, xval)
  local i, j, n, L, poly;
  n := nops(X); poly := 0;
  for i from 1 to n do
    L := 1;
    for j from 1 to n do
      if i ≠ j then
        L := L*(xval − X[j])/(X[i]−X[j]);
      end if;
    end do;
    poly := poly + Y[i]*L;
  end do;
  poly;
end proc:
Px := x → Lagrange(Xvals, Yvals, x) :
printf("Interpolating polynomial P(x):\n");
expand(Lagrange(Xvals, Yvals, x));
printf("\n");
# Plot f(x), P(x), and the error P(x)-f(x)
with(plots) :
plot([evalf(f(x)), Px(x), evalf(Px(x)−f(x))], x = 0 ..Pi/2,
   color = [red, blue, green],
   legend = ["sin(x)", "P(x)", "P(x)-sin(x)"]);
# Evaluate P(x) at pi/3 and pi/4, compute actual errors
xvals := [evalf(Pi/3), evalf(Pi/4)] :
for i in xvals do
  approx := Px(i);
  exact := evalf(f(i));
  err := abs(approx − exact);
  printf("P(%.5f) = %.5f, f(%.5f) = %.5f, |error| = %.5f\n", i, approx, i, exact, err);
end do:
# Best error bound using Lagrange remainder formula
# f^(3)(x) for 2nd-degree polynomial
f3 := diff(f(x), x$3) :
# Find max |f^(3)(x)| on [0, Pi/2] by numeric sampling
maxf3 := 0 :
for i from 0 to 100 do
  xi := i*Pi/200; # 101 points from 0 to Pi/2
  val := evalf(abs(subs(x = xi, f3)));
  if val > maxf3 then maxf3 := val; end if;
end do:
# Compute omega(x) = (x-x0)(x-x1)(x-x2)
omega := (x − Xvals[1])*(x − Xvals[2])*(x − Xvals[3]) :
# Find max |omega(x)| on [0, Pi/2] by numeric sampling
maxomega := 0 :
for i from 0 to 100 do
  xi := i*Pi/200;
  val := evalf(abs(subs(x = xi, omega)));
  if val > maxomega then maxomega := val; end if;
end do:
n := 2 :
ErrorBound := evalf(maxf3/factorial(n + 1)*maxomega) :
printf("Best possible error bound ≈ %.6f\n", ErrorBound);
```

Interpolating polynomial P(x):

$$-0.3039635508\,x^2 + 0.3546241426\,\pi x$$

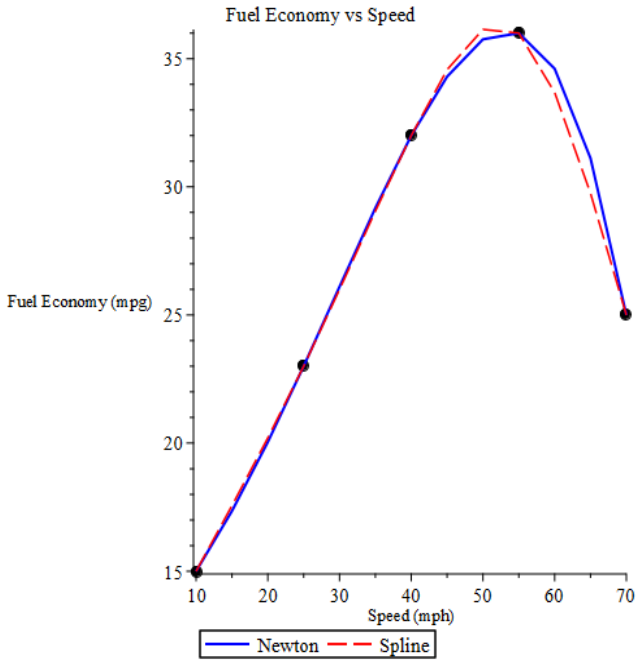


P(1.04720) = 0.83333, f(1.04720) = 0.86603, |error| = 0.03269
P(0.78540) = 0.68750, f(0.78540) = 0.70711, |error| = 0.01961
Best possible error bound ≈ 0.050542

Problem 4

```
# Given data
x := [10, 25, 40, 55, 70];
y := [15, 23, 32, 36, 25];
n := nops(x);
 # Newton's Divided Differences
DD := [seq([y[i]], i=1..n)];

for j from 2 to n do
   for i from 1 to n - j + 1 do
      DD[i] := [op(DD[i]), (DD[i+1][j-1] - DD[i][j-1]) / (x[i+j-1] - x[i])];
   end do;
end do;

printf("Divided Difference Table:\n");
for i from 1 to n do
   for j from 1 to n - i + 1 do
      printf("%10.6f ", DD[i][j]);
   end do;
   printf("\n");
 end do;
 # Newton Polynomial (numeric evaluation)

P := proc(xval)
   local i, val, term;
   val := DD[1][1];
   term := 1;
   for i from 1 to n-1 do
      term := term* (xval - x[i]);
      val := val + DD[1][i+1]* term;
   end do;
   evalf(val);
end proc;

P65 := P(65);
printf("\nEstimated fuel economy (Newton) at 65 mph: %.4f mpg\n", P65);
 # (c) Cubic spline interpolation
with(CurveFitting) :

S := Spline(x, y, xval, degree=3, endpoints='natural') :
S65 := evalf(subs(xval=65, S));
printf("Estimated fuel economy (Spline) at 65 mph: %.4f mpg\n", S65);

# (d) Simple plot (numeric only)

pts := [seq([x[i], y[i]], i=1..n)];
newt_pts := [seq([X, P(X)], X in [seq(10 + 5*i, i=0..12)])];
spline_pts := [seq([X, evalf(subs(xval=X, S))], X in [seq(10 + 5*i, i=0..12)])];

p1 := pointplot(pts, symbol=solidcircle, symbolsize=14, color=black) :
p2 := plot(newt_pts, color=blue, thickness=2, legend="Newton") :
p3 := plot(spline_pts, color=red, linestyle=3, legend="Spline") :

display([p1, p2, p3], title="Fuel Economy vs Speed", labels=["Speed (mph)", "Fuel Economy (mpg)"]);
```

```
x := [10, 25, 40, 55, 70]
y := [15, 23, 32, 36, 25]
n := 5
DD := [[15], [23], [32], [36], [25]]

Divided Difference Table:
 15.000000   0.533333   0.002222  -0.000296  -0.000003
 23.000000   0.600000  -0.011111  -0.000494
 32.000000   0.266667  -0.033333
 36.000000  -0.733333
 25.000000

P := proc(xval) local i, val, term; val := DD[1][1]; term := 1; for i to n - 1 do term := term*(xval - x[i]); val := val + DD[1][i+1]*term end do; evalf(val) end proc
P65 := 31.11522634

Estimated fuel economy (Newton) at 65 mph: 31.1152 mpg

S65 := 29.74603175

Estimated fuel economy (Spline) at 65 mph: 29.7460 mpg

pts := [[10, 15], [25, 23], [40, 32], [55, 36], [70, 25]]
newt_pts := [[10, 15.], [15, 17.34979424], [20, 20.04115226], [25, 23.], [30, 26.10288066], [35, 29.17695473], [40, 32.], [45, 34.30041152], [50, 35.75720165], [55, 36.], [60, 34.60905350], [65, 31.11522634], [70, 25.]]
spline_pts := [[10, 15.], [15, 17.56084656], [20, 20.20105820], [25, 23.], [30, 25.99470899], [35, 29.05291005], [40, 32.], [45, 34.57142857], [50, 36.14285714], [55, 36.], [60, 33.68253968], [65, 29.74603175], [70, 25.]]
```



a) the fuel economy will be 31.12mpg when using Newtons DD
b) Using the natural cubic spline method it should be 29.75
c)The spline doesnt seem to overshoot as much as Newtons DD method seems to so it would be better go to with the cubic spline

Problem 5

```
# Define symbolic nodes and function values
 n := 3 : # number of intervals
 x := 'x':
X := [seq(cat(x, i), i = 0 ..n)] :# symbolic nodes: x0, x1, ..., xn
f := [seq(cat(f, i), i = 0 ..n)] : # symbolic function values: f0, f1, ..., fn
g := 0 :
for i from 1 to n do
  term := f[i];
  for j from 1 to n do
    if j ≠ i then
      term := term * (x − X[j]) / (X[i] − X[j]);
    end if;
  end do;
  g := g + term;
end do;
g := simplify(g);
h := 0 :
for i from 2 to n + 1 do
  term := f[i];
  for j from 2 to n + 1 do
    if j ≠ i then
      term := term * (x − X[j]) / (X[i] − X[j]);
    end if;
  end do;
  h := h + term;
end do;
h := simplify(h);
 #Defining P(x)
P := simplify(g + (X[1] − x) / (X[n + 1] − X[1]) * (g − h));
for i from 1 to n + 1 do
  val := simplify(subs(x=X[i], P)); # simplify to show exactly f_i
  printf("At x = %a:  P(x) = %a   expected = %a\n",
      X[i], val, f[i]);
end do;
```

$$term := [f0,f1,f2,f3]0$$

$$g := \frac{[f0,f1,f2,f3]0\,(x-x1)\,(x-x2)}{(x0-x1)\,(x0-x2)}$$

$$term := [f0,f1,f2,f3]1$$

$$g := \frac{[f0,f1,f2,f3]0\,(x-x1)\,(x-x2)}{(x0-x1)\,(x0-x2)} + \frac{[f0,f1,f2,f3]1\,(x-x0)\,(x-x2)}{(x1-x0)\,(x1-x2)}$$

$$term := [f0,f1,f2,f3]2$$

$$g := \frac{[f0,f1,f2,f3]0\,(x-x1)\,(x-x2)}{(x0-x1)\,(x0-x2)} + \frac{[f0,f1,f2,f3]1\,(x-x0)\,(x-x2)}{(x1-x0)\,(x1-x2)} + \frac{[f0,f1,f2,f3]2\,(x-x0)\,(x-x1)}{(x2-x0)\,(x2-x1)}$$

$$g := \frac{[f0,f1,f2,f3]0\,(x-x1)\,(x-x2)}{(x0-x1)\,(x0-x2)} + \frac{[f0,f1,f2,f3]1\,(x-x0)\,(x-x2)}{(x1-x0)\,(x1-x2)} + \frac{[f0,f1,f2,f3]2\,(x-x0)\,(x-x1)}{(x2-x0)\,(x2-x1)}$$

$$term := [f0,f1,f2,f3]1$$

$$h := \frac{[f0,f1,f2,f3]1\,(x-x2)\,(x-x3)}{(x1-x2)\,(x1-x3)}$$

$$term := [f0,f1,f2,f3]2$$

$$h := \frac{[f0,f1,f2,f3]1\,(x-x2)\,(x-x3)}{(x1-x2)\,(x1-x3)} + \frac{[f0,f1,f2,f3]2\,(x-x1)\,(x-x3)}{(x2-x1)\,(x2-x3)}$$

$$term := [f0,f1,f2,f3]3$$

$$h := \frac{[f0,f1,f2,f3]1\,(x-x2)\,(x-x3)}{(x1-x2)\,(x1-x3)} + \frac{[f0,f1,f2,f3]2\,(x-x1)\,(x-x3)}{(x2-x1)\,(x2-x3)} + \frac{[f0,f1,f2,f3]3\,(x-x1)\,(x-x2)}{(x3-x1)\,(x3-x2)}$$

$$h := \frac{[f0,f1,f2,f3]1\,(x-x2)\,(x-x3)}{(x1-x2)\,(x1-x3)} + \frac{[f0,f1,f2,f3]2\,(x-x1)\,(x-x3)}{(x2-x1)\,(x2-x3)} + \frac{[f0,f1,f2,f3]3\,(x-x1)\,(x-x2)}{(x3-x1)\,(x3-x2)}$$

$$P := \frac{[f0,f1,f2,f3]0\,(x-x1)\,(x-x2)}{(x0-x1)\,(x0-x2)} + \frac{[f0,f1,f2,f3]1\,(x-x0)\,(x-x2)}{(x1-x0)\,(x1-x2)} + \frac{[f0,f1,f2,f3]2\,(x-x0)\,(x-x1)}{(x2-x0)\,(x2-x1)} + \frac{1}{x3-x0}\left((x0-x)\left(\frac{[f0,f1,f2,f3]0\,(x-x1)\,(x-x2)}{(x0-x1)\,(x0-x2)} + \frac{[f0,f1,f2,f3]1\,(x-x0)\,(x-x2)}{(x1-x0)\,(x1-x2)} + \frac{[f0,f1,f2,f3]2\,(x-x0)\,(x-x1)}{(x2-x0)\,(x2-x1)} - \frac{[f0,f1,f2,f3]1\,(x-x2)\,(x-x3)}{(x1-x2)\,(x1-x3)} - \frac{[f0,f1,f2,f3]2\,(x-x1)\,(x-x3)}{(x2-x1)\,(x2-x3)} - \frac{[f0,f1,f2,f3]3\,(x-x1)\,(x-x2)}{(x3-x1)\,(x3-x2)}\right)\right)$$

$$val := [f0,f1,f2,f3]0$$

```
At x = x0:  P(x) = `[f0, f1, f2, f3]0`   expected = `[f0, f1, f2, f3]0`
```

$$val := [f0,f1,f2,f3]1$$

```
At x = x1:  P(x) = `[f0, f1, f2, f3]1`   expected = `[f0, f1, f2, f3]1`
```

$$val := [f0,f1,f2,f3]2$$

```
At x = x2:  P(x) = `[f0, f1, f2, f3]2`   expected = `[f0, f1, f2, f3]2`
```

$$val := [f0,f1,f2,f3]3$$

```
At x = x3:  P(x) = `[f0, f1, f2, f3]3`   expected = `[f0, f1, f2, f3]3`
```

Project 6

```
# Define f(x) and derivative
f := x -> 9/(x^2 + 9):
df := diff(f(x), x):
X := [-3, -1, 1, 3]:
n := nops(X) - 1:
# Build Hermite nodes and repeated values
Z := []:
F := []:
for i from 1 to n + 1 do
  Z := [op(Z), X[i], X[i]]:      # repeat each node
  F := [op(F), f(X[i]), f(X[i])]: # repeat function values
end do:
N := nops(Z):
# Divided differences table
DD := Array(1..N, 1..N, fill = 0):
for i from 1 to N do
  DD[i, 1] := F[i]:
end do:
for j from 2 to N do
  for i from 1 to N - j + 1 do
    if Z[i] = Z[i + j - 1] then
      DD[i, j] := evalf(subs(x = Z[i], df) / factorial(j - 1)):
    else
      DD[i, j] := evalf((DD[i + 1, j - 1] - DD[i, j - 1]) / (Z[i + j - 1] - Z[i])):
    end if:
  end do:
end do:
# Hermite polynomial as function
H := proc(xx)
  local i, j, term, val:
  val := DD[1, 1]:
  for i from 1 to N - 1 do
    term := 1:
    for j from 1 to i do
      term := term * (xx - Z[j]):
    end do:
    val := val + DD[1, i + 1] * term:
  end do:
  evalf(val):
end proc:
# Print Hermite polynomial symbolically
Hpoly := DD[1, 1]:
for i from 1 to N - 1 do
  term := 1:
  for j from 1 to i do
    term := term * (x - Z[j]):
  end do:
```

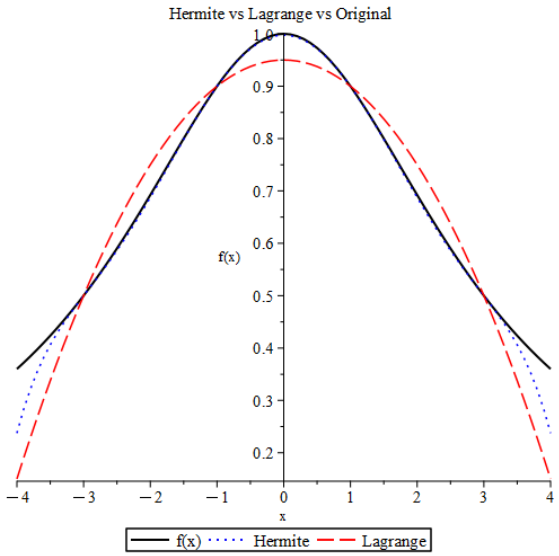
```
    Hpoly := Hpoly + DD[1, i + 1] * term :
  end do:
  Hpoly := expand(Hpoly) :
  print("Hermite Polynomial H(x) = ", Hpoly) :
  # Evaluate H(x) at test points
  xtest := [-2, 4] :
  for xi in xtest do
    Hval := H(xi) :
    err := abs(Hval - f(xi)) :
    printf("x = %d, H(x) = %.6f, f(x) = %.6f, error = %.6f\n", xi, Hval, f(xi), err) :
  end do:
  # Lagrange polynomial as function
  L := proc(xx)
    local i, j, term, val :
    val := 0 :
    for i from 1 to n + 1 do
      term := f(X[i]) :
      for j from 1 to n + 1 do
        if j ≠ i then
          term := term * (xx - X[j]) / (X[i] - X[j]) :
        end if:
      end do:
      val := val + term :
    end do:
    evalf(val) :
  end proc:
  # Print Lagrange polynomial symbolically
  Ppoly := 0 :
  for i from 1 to n + 1 do
    term := f(X[i]) :
    for j from 1 to n + 1 do
      if j ≠ i then
        term := term * (x - X[j]) / (X[i] - X[j]) :
      end if:
    end do:
    Ppoly := Ppoly + term :
  end do:
  Ppoly := expand(Ppoly) :
  print("Lagrange Polynomial P(x) = ", Ppoly) :
  # Evaluate Lagrange at same points
  for xi in xtest do
    Lval := L(xi) :
    errL := abs(Lval - f(xi)) :
    printf("x = %d, P(x) = %.6f, f(x) = %.6f, error = %.6f\n", xi, Lval, f(xi), errL) :

end do:
# Plot original f(x), Hermite, and Lagrange
with(plots) :
xxvals := [seq(-4 + 0.1 * i, i = 0 ..80)] :
f_pts := [seq([xx, f(xx)], xx in xxvals)] :
H_pts := [seq([xx, H(xx)], xx in xxvals)] :
L_pts := [seq([xx, L(xx)], xx in xxvals)] :
p1 := plot(f_pts, color = black, thickness = 2, legend = "f(x)") :
p2 := plot(H_pts, color = blue, linestyle = 2, legend = "Hermite") :
p3 := plot(L_pts, color = red, linestyle = 3, legend = "Lagrange") :
display([p1, p2, p3], title = "Hermite vs Lagrange vs Original", labels = ["x", "f(x)"]);
```

"Hermite Polynomial H(x) = ", $-0.0002777777780\,x^6 + 0.9974999999 + 0.008055555564\,x^4 + 1.\times 10^{-11}\,x^3 - 0.1052777778\,x^2$

```
x = -2, H(x) = 0.687500, f(x) = 0.692308, error = 0.004808
x = 4, H(x) = 0.237500, f(x) = 0.360000, error = 0.122500
```

"Lagrange Polynomial P(x) = ", $-\frac{x^2}{20} + \frac{19}{20}$

```
x = -2, P(x) = 0.750000, f(x) = 0.692308, error = 0.057692
x = 4, P(x) = 0.150000, f(x) = 0.360000, error = 0.210000
```



Here the hermite obviously more closely follows f(x) nearly hiding under the f(x) curve for almost the entire graph.

Problem 7

```
# Define the spline pieces symbolically
s1 := x -> x^3 + 2*x^2 + 1;          # 1 <= x <= 2
s2 := x -> -2*x^3 + beta*x^2 - 36*x + 25; # 2 ≤ x ≤ 3
 # Solve for beta using continuity at x=2
# Function continuity
eq1 := s1(2) = s2(2);
# First derivative continuity
s1p := diff(s1(x), x);
s2p := diff(s2(x), x);
eq2 := subs(x = 2, s1p) = subs(x = 2, s2p);
# Second derivative continuity
s1pp := diff(s1(x), x$2);
s2pp := diff(s2(x), x$2);
eq3 := subs(x = 2, s1pp) = subs(x = 2, s2pp);
# Solve for beta
sol := solve( {eq1, eq2, eq3}, beta);
# Extract numeric beta
for eq in sol do
   beta_val := rhs(eq);
end do;
 beta_val; # Maple prints numeric beta
 # Evaluate second derivatives at endpoints
dd_left := eval(subs(x = 1, s1pp) );
dd_right := eval(subs( {x = 3, beta = beta_val}, s2pp) );
dd_left;
 dd_right;
# Check if it is a natural cubic spline
if dd_left = 0 and dd_right = 0 then
   "It IS a natural cubic spline";
else
   "It is NOT a natural cubic spline";
 end if;
 # Define s2 as fully numeric for plotting
 s2_num := x -> -2*x^3 + beta_val*x^2 - 36*x + 25;
 # Plot the spline pieces cleanly
with(plots) :
p1 := plot(s1(x), x = 1 ..2, color = blue, thickness = 2, legend = "s1(x)" );
p2 := plot(s2_num(x), x = 2 ..3, color = red, thickness = 2, legend = "s2(x)" );
display( [p1, p2], title = "Cubic Spline", labels = ["x", "s(x)" ]);
```

$$s1 := x \mapsto x^3 + 2 \cdot x^2 + 1$$

$$s2 := x \mapsto -2 \cdot x^3 + \beta \cdot x^2 - 36 \cdot x + 25$$

$$eq1 := 17 = -63 + 4\,\beta$$

$$s1p := 3\,x^2 + 4\,x$$

$$s2p := 2\,\beta\,x - 6\,x^2 - 36$$

$$eq2 := 20 = 4\,\beta - 60$$

$$s1pp := 6\,x + 4$$

$$s2pp := 2\,\beta - 12\,x$$

$$eq3 := 16 = 2\,\beta - 24$$

$$sol := \{\beta = 20\}$$

$$beta\_val := 20$$

20

$$dd\_left := 10$$

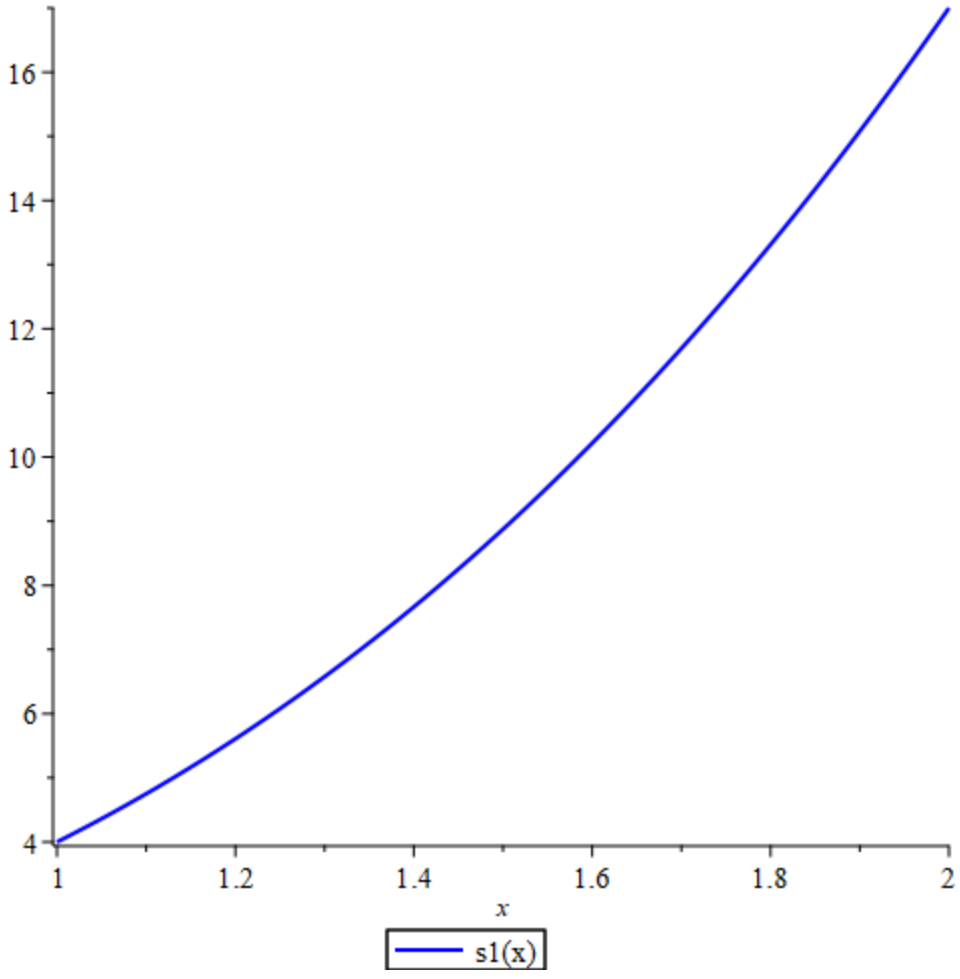
$$dd\_right := 4$$

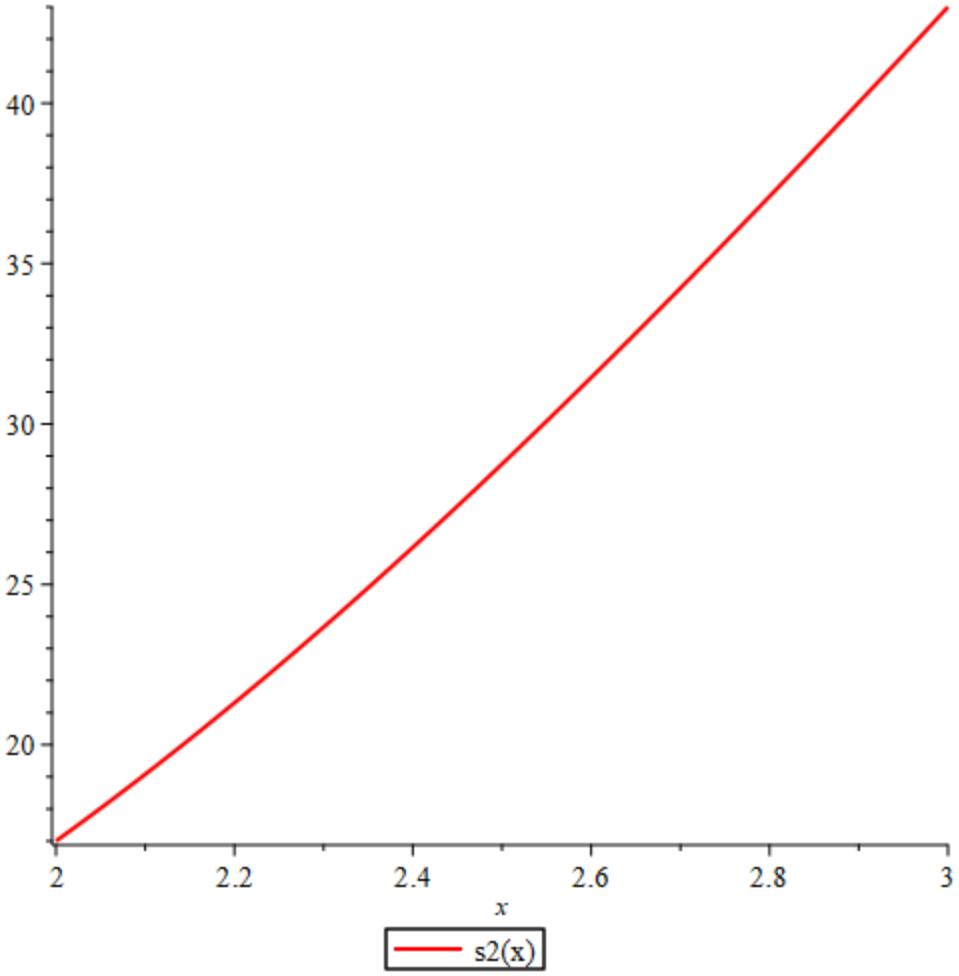
10

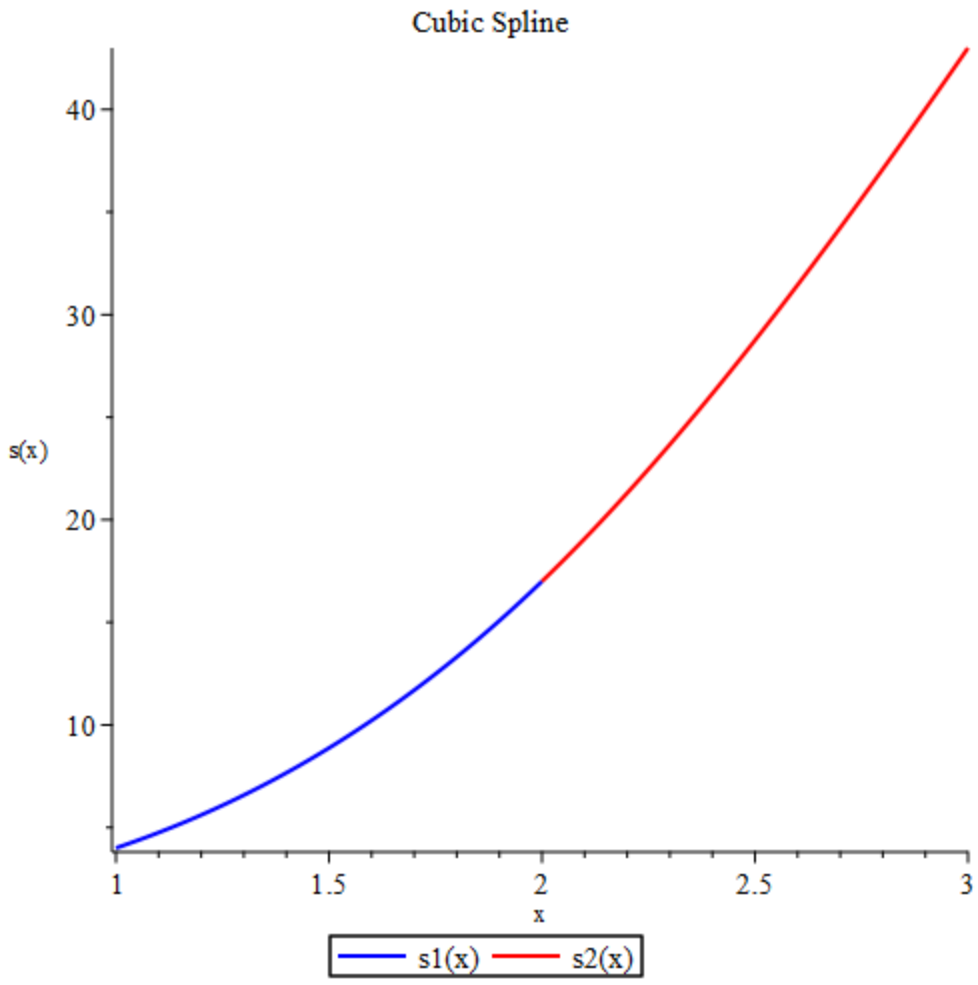
4

"It is NOT a natural cubic spline"

$$s2\_mm := x \mapsto -2 \cdot x^3 + beta\_val \cdot x^2 - 36 \cdot x + 25$$




40
35
30
25
20
2
2.2
2.4
2.6
2.8
3
x
s2(x)



This is not a natural cubic spline due to the 2nd derivatives at the end points not being zero

Problem 8

```
Spline := proc(x::list, y::list, kind::string, slope_start::numeric := 0, slope_end::numeric := 0)
  local n, h, alpha, l, mu, z, c, b, d, pieces, i;
  n := nops(x);
  if n < 2 then error "Need at least two data points"; end if;
  h := [seq(x[i + 1] - x[i], i = 1 .. n - 1)];
  alpha := Array(1 .. n, fill = 0);
  if kind = "clamped" then
    alpha[1] := 3*(y[2] - y[1])/h[1] - 3*slope_start;
    alpha[n] := 3*slope_end - 3*(y[n] - y[n - 1])/h[n - 1];
  end if;
  for i from 2 to n - 1 do
    alpha[i] := 3*(y[i + 1] - y[i])/h[i] - 3*(y[i] - y[i - 1])/h[i - 1];
  end do;
  l := Array(1 .. n); mu := Array(1 .. n); z := Array(1 .. n);
  l[1] := 1; mu[1] := 0; z[1] := 0;
  for i from 2 to n - 1 do
    l[i] := 2*(x[i + 1] - x[i - 1]) - h[i - 1]*mu[i - 1];
    mu[i] := h[i]/l[i];
    z[i] := (alpha[i] - h[i - 1]*z[i - 1])/l[i];
  end do;
  if kind = "clamped" then
    l[n] := h[n - 1]*(2 - mu[n - 1]);
    z[n] := (alpha[n] - h[n - 1]*z[n - 1])/l[n];
  else
    l[n] := 1; z[n] := 0;
  end if;
  c := Array(1 .. n, fill = 0); b := Array(1 .. n - 1); d := Array(1 .. n - 1);
  if kind = "clamped" then c[n] := z[n]; end if;
  for i from n - 1 by -1 to 1 do
    c[i] := z[i] - mu[i]*c[i + 1];
    b[i] := (y[i + 1] - y[i])/h[i] - h[i]*(2*c[i] + c[i + 1])/3;
    d[i] := (c[i + 1] - c[i])/(3*h[i]);
  end do;
  pieces := [seq(unapply(y[i] + b[i]*(t - x[i]) + c[i]*(t - x[i])^2 + d[i]*(t - x[i])^3, t), i = 1 .. n - 1)];
  return pieces;
end proc:
AddPieces := proc(pieces, x, colorname::string)
  local i;
  plots:-display([seq(plot(pieces[i](t), t = x[i] .. x[i + 1], color = colorname, thickness = 3), i = 1 .. nops(pieces))]);
end proc:
LagrangeSegment := proc(x::list, y::list)
  local n, i, j, L, term;
  n := nops(x);
  L := 0;
  for i from 1 to n do
    term := y[i];
    for j from 1 to n do
```

```
      for j from 1 to n do
        if j ≠ i then
          term := term* (t − x[j]) / (x[i] − x[j]);
        end if;
      end do;
      L := L + term;
    end do;
    return unapply(L, t);
  end proc:
  # Data
x1 := [1, 2, 4, 5, 7, 8, 10, 13, 17]; y1 := [3.0, 3.7, 3.9, 4.2, 5.7, 6.6, 7.1, 6.7, 4.5]; fp0_1 := 1.0; fpn_1 := −0.67;
x2 := [17, 20, 23, 24, 25, 27, 27.7]; y2 := [4.5, 7.0, 6.1, 5.6, 5.8, 5.2, 4.1]; fp0_2 := 3.0; fpn_2 := −4.0;
x3 := [27.7, 28, 29, 30]; y3 := [4.1, 4.3, 4.1, 3.0]; fp0_3 := 0.33; fpn_3 := −1.5;
  # Compute splines
Clamped1 := Spline(x1, y1, "clamped", fp0_1, fpn_1);
Natural1 := Spline(x1, y1, "natural");
Clamped2 := Spline(x2, y2, "clamped", fp0_2, fpn_2);
Natural2 := Spline(x2, y2, "natural");
Clamped3 := Spline(x3, y3, "clamped", fp0_3, fpn_3);
Natural3 := Spline(x3, y3, "natural");
  # Compute Lagrange per segment
Lagr1 := LagrangeSegment(x1, y1);
Lagr2 := LagrangeSegment(x2, y2);
Lagr3 := LagrangeSegment(x3, y3);
  # Display
display(
    [
      AddPieces(Clamped1, x1, "red"),
      AddPieces(Natural1, x1, "blue"),
      plot(Lagr1(t), t = x1[1] .. x1[−1], color = "green", linestyle = 3, thickness = 2),
      AddPieces(Clamped2, x2, "red"),
      AddPieces(Natural2, x2, "blue"),
      plot(Lagr2(t), t = x2[1] .. x2[−1], color = "green", linestyle = 3, thickness = 2),
      AddPieces(Clamped3, x3, "red"),
      AddPieces(Natural3, x3, "blue"),
      plot(Lagr3(t), t = x3[1] .. x3[−1], color = "green", linestyle = 3, thickness = 2)
    ],
    title = "Snoopy Curve: Clamped (red), Natural (blue), Piecewise Lagrange (green dotted)",
    labels = ["x", "y"]
  );
```

,,

$$x1 := [1, 2, 4, 5, 7, 8, 10, 13, 17]$$
$$y1 := [3.0, 3.7, 3.9, 4.2, 5.7, 6.6, 7.1, 6.7, 4.5]$$
$$fp0\_1 := 1.0$$
$$fpn\_1 := -0.67$$
$$x2 := [17, 20, 23, 24, 25, 27, 27.7]$$
$$y2 := [4.5, 7.0, 6.1, 5.6, 5.8, 5.2, 4.1]$$
$$fp0\_2 := 3.0$$
$$fpn\_2 := -4.0$$
$$x3 := [27.7, 28, 29, 30]$$
$$y3 := [4.1, 4.3, 4.1, 3.0]$$
$$fp0\_3 := 0.33$$
$$fpn\_3 := -1.5$$

$$Clamped1 := [t \mapsto 2.178503063 + 0.8214969370 \cdot t - 0.1214969370 \cdot (t-1)^3, t \mapsto 2.785987748 + 0.4570061260 \cdot t - 0.3644908109 \cdot (t-2)^2 + 0.09299387393 \cdot (t-2)^3, t \mapsto 3.440122521 + 0.1149693697 \cdot t + 0.1934724327 \cdot (t-4)^2 - 0.008441802367 \cdot (t-4)^3, t \mapsto 1.817055860 + 0.4765888280 \cdot t + 0.1681470256 \cdot (t-5)^2 - 0.01572071981 \cdot (t-5)^3, t \mapsto -1.023698050 + 0.9605282928 \cdot t + 0.07382270677 \cdot (t-7)^2 - 0.1343509996 \cdot (t-7)^3, t \mapsto 0.959034342 + 0.7051207073 \cdot t - 0.3292302919 \cdot (t-8)^2 + 0.05083496907 \cdot (t-8)^3, t \mapsto 7.117808312 - 0.0017808312 \cdot t - 0.02422047742 \cdot (t-10)^2 - 0.006543452207 \cdot (t-10)^3, t \mapsto 10.90909977 - 0.3237769053 \cdot t - 0.08311154729 \cdot (t-13)^2 + 0.006638943410 \cdot (t-13)^3]$$

$$Natural1 := [t \mapsto 2.178502905 + 0.8214970951 \cdot t - 0.1214970951 \cdot (t-1)^3, t \mapsto 2.785988381 + 0.4570058097 \cdot t - 0.3644912854 \cdot (t-2)^2 + 0.09299419027 \cdot (t-2)^3, t \mapsto 3.440116195 + 0.1149709512 \cdot t + 0.1934738561 \cdot (t-4)^2 - 0.008444807333 \cdot (t-4)^3, t \mapsto 1.817078792 + 0.4765842415 \cdot t + 0.1681394341 \cdot (t-5)^2 - 0.01571577741 \cdot (t-5)^3, t \mapsto -1.023868542 + 0.9605526489 \cdot t + 0.07384476963 \cdot (t-7)^2 - 0.1343974185 \cdot (t-7)^3, t \mapsto 0.959600542 + 0.7050499323 \cdot t - 0.3293474860 \cdot (t-8)^2 + 0.05091125987 \cdot (t-8)^3, t \mapsto 7.114048929 - 0.0014048929 \cdot t - 0.02387992666 \cdot (t-10)^2 - 0.006698740053 \cdot (t-10)^3, t \mapsto 10.93215565 - 0.3255504343 \cdot t - 0.08416858713 \cdot (t-13)^2 + 0.007014048927 \cdot (t-13)^3]$$

$$Clamped2 := [t \mapsto -14.25604329 + 1.103296664 \cdot t - 0.02999592563 \cdot (t-17)^3, t \mapsto 1.131866560 + 0.2934066720 \cdot t - 0.2699633307 \cdot (t-20)^2 + 0.02405370224 \cdot (t-20)^3, t \mapsto 21.66923709 - 0.6769233517 \cdot t - 0.05348001052 \cdot (t-23)^2 + 0.2304033622 \cdot (t-23)^3, t \mapsto 7.824158862 - 0.0926732859 \cdot t + 0.6377300762 \cdot (t-24)^2 - 0.3450567903 \cdot (t-24)^3, t \mapsto 2.109587608 + 0.1476164957 \cdot t - 0.3974402944 \cdot (t-25)^2 + 0.08681602327 \cdot (t-25)^3, t \mapsto 16.00951488 - 0.400352403 \cdot t + 0.1234558452 \cdot (t-27)^2 - 2.566316857 \cdot (t-27)^3]$$

$$Natural2 := [t \mapsto -14.29748443 + 1.105734378 \cdot t - 0.03026678269 \cdot (t-17)^3, t \mapsto 1.229375106 + 0.2885312447 \cdot t - 0.2724010442 \cdot (t-20)^2 + 0.02540798756 \cdot (t-20)^3, t \mapsto 21.27676520 - 0.6598593567 \cdot t - 0.04372915623 \cdot (t-23)^2 + 0.2035885129 \cdot (t-23)^3, t \mapsto 8.877251120 - 0.1365521300 \cdot t + 0.5670363826 \cdot (t-24)^2 - 0.2304842524 \cdot (t-24)^3, t \mapsto -1.851696942 + 0.3060678777 \cdot t - 0.1244163746 \cdot (t-25)^2 - 0.08930878213 \cdot (t-25)^3, t \mapsto 39.30918116 - 1.263303006 \cdot t + 0.6602690675 \cdot (t-27)^2 + 0.3144138417 \cdot (t-27)^3]$$

$$Clamped3 := [t \mapsto -16.76049383 + 0.7530864198 \cdot t - 0.9602194787 \cdot (t-27.7)^3, t \mapsto -9.52716050 + 0.4938271607 \cdot t - 0.8641975309 \cdot (t-28)^2 + 0.1703703704 \cdot (t-28)^3, t \mapsto 25.08024691 - 0.7234567900 \cdot t - 0.3530864198 \cdot (t-29)^2 - 0.02345679013 \cdot (t-29)^3]$$

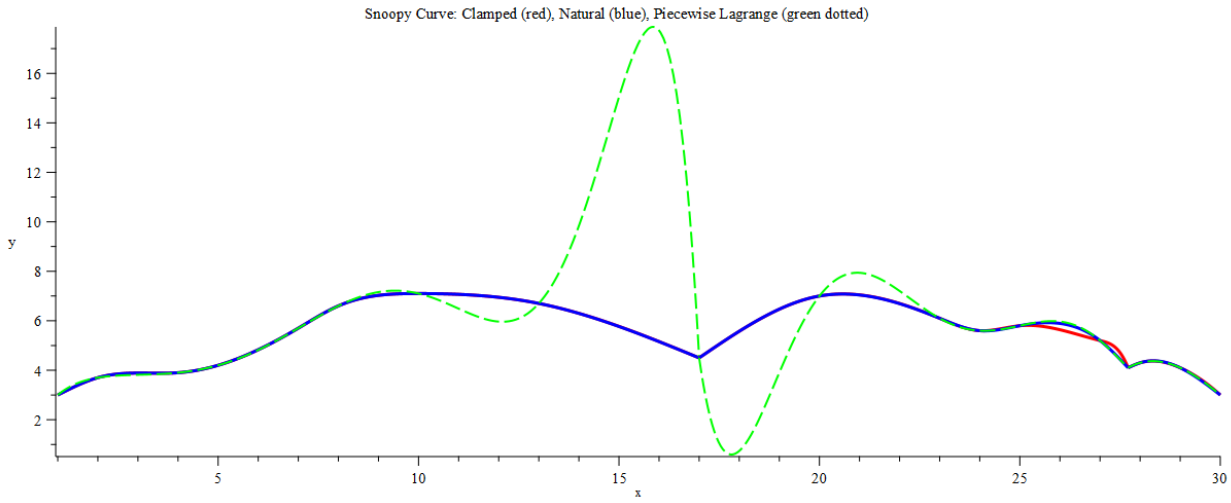
$$Natural3 := [t \mapsto -16.63570922 + 0.7485815603 \cdot t - 0.9101654847 \cdot (t-27.7)^3, t \mapsto -9.77943262 + 0.5028368793 \cdot t - 0.8191489362 \cdot (t-28)^2 + 0.1163120567 \cdot (t-28)^3, t \mapsto 26.90921986 - 0.7865248227 \cdot t - 0.4702127660 \cdot (t-29)^2 + 0.1567375887 \cdot (t-29)^3]$$

$$Lagr1 := t \mapsto -\frac{(-3.0 \cdot t + 6.0) \cdot (t-4) \cdot (t-5) \cdot (t-7) \cdot (t-8) \cdot (t-10) \cdot (t-13) \cdot (t-17)}{870912} - \frac{(3.7 \cdot t - 3.7) \cdot (t-4) \cdot (t-5) \cdot (t-7) \cdot (t-8) \cdot (t-10) \cdot (t-13) \cdot (t-17)}{237600} + \frac{(1.300000000 \cdot t - 1.300000000) \cdot (t-2) \cdot (t-5) \cdot (t-7) \cdot (t-8) \cdot (t-10) \cdot (t-13) \cdot (t-17)}{16848}$$
$$- \frac{(1.050000000 \cdot t - 1.050000000) \cdot (t-2) \cdot (t-4) \cdot (t-7) \cdot (t-8) \cdot (t-10) \cdot (t-13) \cdot (t-17)}{8640} + \frac{(0.9500000000 \cdot t - 0.9500000000) \cdot (t-2) \cdot (t-4) \cdot (t-5) \cdot (t-8) \cdot (t-10) \cdot (t-13) \cdot (t-17)}{5400}$$
$$- \frac{(0.9428571429 \cdot t - 0.9428571429) \cdot (t-2) \cdot (t-4) \cdot (t-5) \cdot (t-7) \cdot (t-10) \cdot (t-13) \cdot (t-17)}{6480} + \frac{(0.7888888889 \cdot t - 0.7888888889) \cdot (t-2) \cdot (t-4) \cdot (t-5) \cdot (t-7) \cdot (t-8) \cdot (t-13) \cdot (t-17)}{30240}$$
$$- \frac{(0.5583333333 \cdot t - 0.5583333333) \cdot (t-2) \cdot (t-4) \cdot (t-5) \cdot (t-7) \cdot (t-8) \cdot (t-10) \cdot (t-17)}{285120} + \frac{(0.2812500000 \cdot t - 0.2812500000) \cdot (t-2) \cdot (t-4) \cdot (t-5) \cdot (t-7) \cdot (t-8) \cdot (t-10) \cdot (t-13)}{5896800}$$

$$Lagr2 := t \mapsto -0.00002781486426 \cdot (-1.500000000 \cdot t + 30.00000000) \cdot (t-23) \cdot (t-24) \cdot (t-25) \cdot (t-27) \cdot (t-27.7) - 0.0003092145949 \cdot (2.333333333 \cdot t - 39.66666666) \cdot (t-23) \cdot (t-24) \cdot (t-25) \cdot (t-27) \cdot (t-27.7) + 0.008865248228 \cdot (1.016666667 \cdot t - 17.28333334) \cdot (t-20) \cdot (t-24) \cdot (t-25) \cdot (t-27) \cdot (t-27.7) - 0.02252252252 \cdot (0.8000000000 \cdot t - 13.60000000) \cdot (t-20) \cdot (t-23) \cdot (t-25) \cdot (t-27) \cdot (t-27.7) + 0.01851851852 \cdot (0.7250000000 \cdot t - 12.32500000) \cdot (t-20) \cdot (t-23) \cdot (t-24) \cdot (t-27) \cdot (t-27.7) - 0.008503401360 \cdot (0.5200000000 \cdot t - 8.840000000) \cdot (t-20) \cdot (t-23) \cdot (t-24) \cdot (t-25) \cdot (t-27.7) + 0.003951371733 \cdot (0.3831775701 \cdot t - 6.514018692) \cdot (t-20) \cdot (t-23) \cdot (t-24) \cdot (t-25) \cdot (t-27)$$

$$Lagr3 := t \mapsto 0.3344481605 \cdot (-13.66666667 \cdot t + 382.6666668) \cdot (t-29) \cdot (t-30) + \frac{(14.33333333 \cdot t - 397.0333332) \cdot (t-29) \cdot (t-30)}{2} - (3.153846154 \cdot t - 87.36153847) \cdot (t-28) \cdot (t-30) + \frac{(1.304347826 \cdot t - 36.13043478) \cdot (t-28) \cdot (t-29)}{2}$$

Snoopy Curve: Clamped (red), Natural (blue), Piecewise Lagrange (green dotted)



As one can see the Natural cubic spline fit the best